

Phase 0

Perform the following operations **only once** for each group:

1. Clone the group's repository - `git clone <group-repo-url>`.
2. Add a remote pointing to the `common` repo - `git remote add common https://github.com/isel-leic-ls/2526-2-common.git`.
3. Pull the contents from the `common` repo - `git pull common main`. If necessary, use the `--allow-unrelated-histories` option.
4. Remove the current content of `README.me` and add the number and name of each group member (use the markdown format).
5. Create a commit containing those changes.
6. Push the commits to the group's repository - `git push origin main`.

Correct errors and add tests (multiple group members should coordinate these tasks)

1. Ensure you have an updated repo.
2. Execute the unit tests using Gradle and check if there are errors.
3. Create a project in IntelliJ using the recommendations of the previous classes.
4. Execute the same testes now using the IntelliJ IDE.
5. Understand de code that is is being tested
6. Add more tests to increase coverage.
7. Check, using the Gradle build script at the command line, that all the tests are executed with success.
8. Publish all the changes (if any) to the local and group repository. Make one or more commits for this task.

Database configuration and connectivity test

1. Create a database in the chosen DB Management system (i.e. Postgres) on your machine. Add a table to represent *students*.
2. Make a set of tests to check that your system can read (i.e. `SELECT`), update (i.e. `INSERT` and `UPDATE`) and delete (i.e. `DELETE`) over the previous table, using the JDBC interface. Assign different tests to each member of the group.
3. Publish the changes to the group repo.

Create tag

1. Create a *tag* with the identifier `0.0.0` and publish it to the group repo
 - `git tag -a 0.0.0 -m "Version 0.0.0"`.
 - `git push origin 0.0.0`.
 - Performed only once per

```
git init git commit -m "LS introduction" git branch -M main git remote add origin https://github.com/isel-leic-ls/2526-2-common.git git push -u origin main
```

Phase 1

Goals

The Software Laboratory project consists of the analysis, design and implementation of an information system to manage **house rentals**.

Its development is divided into 4 phases, with incremental requirements published before the beginning of each phase.

Domain

- A *user* is characterized by:
 - a unique number (identifier)
 - a name
 - a unique email
- A *location* represents a geographical entity and is characterized by:
 - a unique number (identifier)
 - a name
 - a type
 - an optional parent location

Locations are organized hierarchically using a parent-child relationship.

Each location may have a parent location, except root locations (e.g., countries).

The type of a location must be one of the following:

- COUNTRY
- REGION
- DISTRICT
- MUNICIPALITY
- LOCALITY

Example hierarchy:

Portugal (COUNTRY)

└─ Lisboa (DISTRICT)

 └─ Oeiras (MUNICIPALITY)

 └─ Paço de Arcos (LOCALITY)

A location may contain other locations, forming a tree structure.

- A *house* is characterized by:
 - a unique number (identifier)
 - a title (short name)
 - a location (identifier)

- a area in square meters
 - a price per night in euros
 - a description
- A *booking* is characterized by:
 - a unique number (identifier)
 - a start date
 - an end date
 - and is associated to a user and a house

NOTE:

All houses are available every day unless they are booked for a given period.

Phase 1 Requirements

Development of the backend service.

The service must expose an HTTP API to frontend client applications.

The API should provide the following functionalities:

User Management

- Create a new user, given the following parameters:
 - **name** – the user's name
 - **email** – the user's unique email

Returns:

- the user token
 - the user identifier
 - Get the details of a user.
-

Location Management

- Create a new location, given:
 - **name**
 - **type**
 - **parentId** (optional)

Returns:

- the location identifier
- Get the details of a location.
- Get the list of child locations of a given location.

- Get the full hierarchical path of a location
(e.g., Portugal → Lisboa → Oeiras → Paço de Arcos).
-

House Management

- Create a new house, given the following parameters:
 - **title** – house title
 - **location** – location identifier
 - **areaSqMt** – area in square meters
 - **pricePerNight** – price per night
 - **description** – short description

Returns the house identifier.

- Get the detailed information of a house.
 - Get a list with all houses.
-

Booking Management

- Create a booking, given the following parameters:
 - **hid** – house identifier
 - **startDate** – booking start date
 - **endDate** – booking end date
 - Get the detailed information of a booking.
 - Get a list with the bookings, given the following parameters:
 - **hid** – house identifier
 - **date interval** – date interval (returns all bookings that are included in date interval)
 - Get a list with the available houses for a given period:
 - **startDate**
 - **endDate**
-

Paging

All Web API **GET** operations that return a sequence should support paging, defined by:

- **limit** – length of the subsequence to return
- **skip** – start position of the subsequence

Example:

GET /houses?skip=10&limit=5 returns houses from position 11 to 15.

Authentication

For all operations that require authentication, a user token must be sent in the Authorization header using a **Bearer Token**.

This token is generated at user creation and consists of a UUID string (use `kotlin.uuid.Uuid` for ID generation).

Students should decide which functionalities require authentication and authorization.

Price Prediction (Optional)

Predict rental price for a house

Predict the rental price of a house based on its characteristics, given:

- `areaSqMt` – area of the house in square meters
- `location` – location
- `nights` – number of nights

Returns:

- `predictedPricePerNight` – estimated nightly price
- `predictedTotalPrice` – estimated total price

This operation does **not** create a booking and does **not** require an existing house in the system.

Note:

Students may implement the price prediction considering only a single location, using the approach developed in the previous phase (e.g., linear regression to estimate the price of a house given its area in square meters).

Alternative approaches may be used, provided they are previously discussed with the teacher.

Non-functional Requirements

The backend must be developed with **Kotlin**.

- HTTP handling must use the **HTTP4K** library
- Body serialization/deserialization must use **kotlinx.serialization**
- Application data must be stored in a **Postgres database**
- Tests run using **in-memory data**, not the database

NOTE: You can see a functional usage example [here](#).

The following non-functional requirements will be highly valued:

- Readability
- Testability, including on machines not belonging to the development team

NOTE: Tests and data stored in database are mandatory in Phase 1.

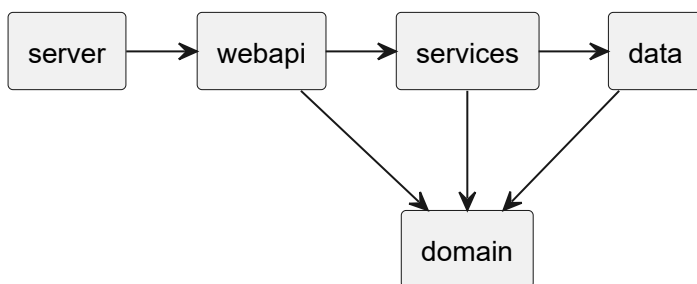
Guidelines for Design and Implementation

At the **initial stage of development**, the server application should maintain its data **in memory**. This approach allows the development process to focus primarily on the **architectural structure, modularization, and implementation of application logic**, before introducing persistent storage mechanisms.

The solution should be organized into the following, each with clearly defined responsibilities:

- `houses.server` – contains the **entry point of the server application**, responsible for initializing and configuring the system.
- `houses.webapi` – responsible for the **implementation of the HTTP API**, including the definition of routes and the handling of HTTP requests and responses.
- `houses.services` – implements the **application's logic**, coordinating operations between the API layer and the data access layer.
- `houses.data` – provides the **data access implementation**, responsible for storing and retrieving application data during execution.
- `houses.domain` – defines the **domain model**, including the domain data structures.

Dependencies:



NOTE: The `domain` package contains the **core domain entities** that are used by the `webapi`, `services`, and `data`.

Report and HTTP API Documentation

The Phase 1 delivery must include:

- A **technical report** in the `docs` folder, using the provided [template](#)
 - An **HTTP API documentation** for frontend usage
 - Must not expose internal implementation details
 - Students are encouraged to follow the **OpenAPI specification**
-

Delivery Date

The project must be delivered until **end of week 5**, via the creation of a **0.1.0 tag** on the GitHub repository.

Phase 2

Introduction

This document describes the requirements for the second phase of the Software Laboratory project.

Requirements

Additional Operations

- Get a list with the bookings of a user
- Delete a booking
- Update a booking
 - `date interval` - booking date interval

In-Memory Cache in server (Optional)

As an optional enhancement, the system may include an in-memory cache to improve performance when retrieving house details.

The cache should store the results of the last N accesses to house details.

- If the information is already in the cache, it should be returned directly from the cache.
- Otherwise, the system should retrieve the information normally and store it in the cache.

The value of N should be configurable.

Note: The proposed caching solution must be discussed with the teacher before implementation.

Single Page Application

The main requirement for the second phase is the delivery of a [Single Page Application \(SPA\)](#) to provide a Web User Interface to the `GET` operations developed in the first phase.

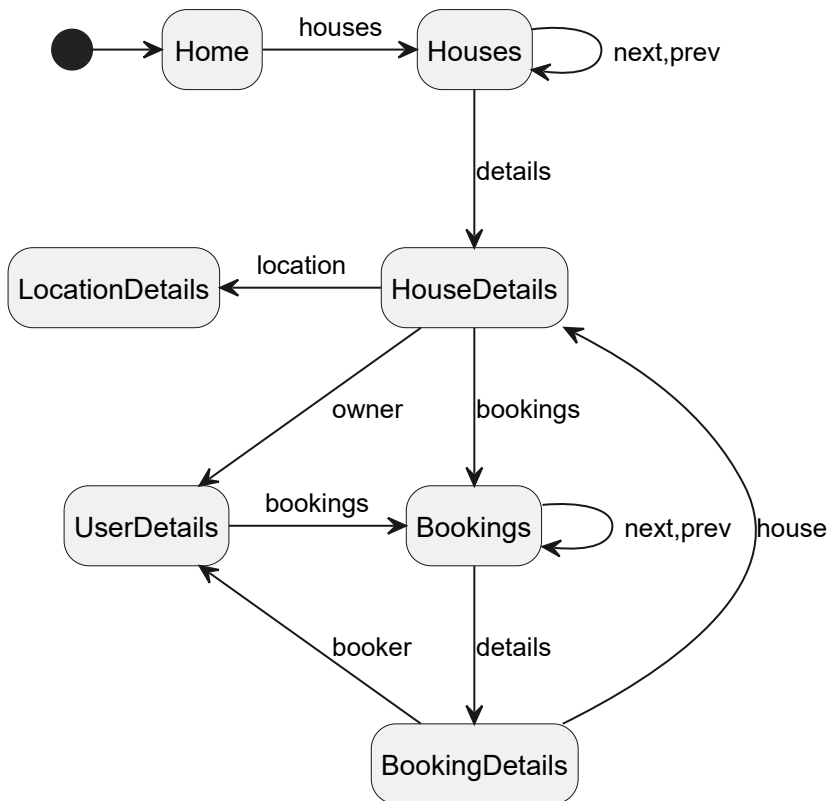
In this repository there are two examples of simple Single Page Applications:

1. [A simple one](#)
2. [One more complete using a router](#)

To see these two examples working, you must launch the HTTPServer application on `file pt.isel.ls.http.HttpServer.kt` and go to the following URI on your web browser:
`http://localhost:9000/`

Note that the routes in `file pt.isel.ls.http.HttpServer.kt` have been updated.

Create HTML views that contain links to ensure the navigability defined in the following graph, changes to the graph should be discuss with the teacher:



Note: All views must have a link to Home

(Click [here](#) for a bigger picture)

To simplify the creation of HTML views, students should implement a small **Domain-Specific Language (DSL)** in JavaScript for building HTML elements.

Instead of repeatedly using low-level DOM APIs such as `document.createElement`, the DSL should provide a more declarative and composable way to construct the UI.

For example, rather than:

```
const div = document.createElement("div")
div.appendChild(document.createTextNode("Hello"))
```

Students could define helper functions such as:

```
div("Hello")
```

or more structured compositions:


```
div(  
  h1("Title"),  
  p("Some text")  
)
```

The goal is to improve readability, reduce boilerplate, and make view construction more expressive.

Report

The technical report created for phase 1 should be updated and/or extended with the relevant technical information. The sections developed for phase 1 can be improved or changed. There should **not** be a separate report for phase 2. The goal is to have a single report, updated through all the project phases.

There isn't any template for phase 2.

Delivery date

The phase 2 must be delivered until 25 of April, through the creation of the 0.2.0 tag on the group's GitHub repository.

Phase 3

Introduction

This document describes the requirements for the third phase of the Software Laboratory project.

Requirements

Additional Operations

- Add an operation to list the available days to book an house in specific month.

Single Page Application

When using the web application, in no situation the (human) user will have to know and/or insert any id.

Views

- Update the Home view to have an option to search for houses.
- Create a new view to show the available days to book a house in a specific month.
- Update House details view to have an option to search for bookings by date interval.
- Add the necessary views to create a location, a house, and a booking. Also add support for delete and update of a given booking.

Note: [Example using HTML form](#)

Refactoring code

This phase should be also used to critically review all code baseline, and start refactoring to improve code quality. Also, more tests should be produced in order to increase the overall coverage.

Report

The technical report created in the previous phases should be updated and/or extended with the relevant technical information. Remember the goal is to have a single report, updated through all the project phases.

Delivery date

The phase 3 must be delivered until the end of week 13 , through the creation of the 0.3.0 tag on the group's GitHub repository.

Phase 4

Introduction

This document describes the requirements for the fourth and final phase of the Software Laboratory project.

Requirements

Additional Operations

- Update the create user operation, to receive a password.
- Create a new operation that receives the user name and password, and returns the user token and id

Single Page Application

- Produce views to create a new user (Sign Up) and to obtain the user's token (Login).
- The received token should be stored until the user closes the browser window/tab or explicitly executes logout.

Hosting on Render

Host the application on the [Render](#).

1. Create a [Render](#) free account. Render is an example of a Platform-as-a-Service (PaaS) provider.
2. On the Render web site, create a new [PostgreSQL instance](#)
 - Give it the **Name**: `db-1s-2526-2-<turma>-g<número-do-grupo>`;
 - You can leave the **Database** and **User** fields empty;
 - Select the "Frankfurt(EU Central)" region;
 - Select the Free instance type, and click **Create Database**;
 - Wait for the creation of the database;
 - Create the tables in the render Postgres database, use psql, pgAdmin or a similar tool.
 - **Note that the connection URL must be in the JDBC format (ex. `jdbc:postgresql://<hostname>.frankfurt-postgres.render.com:<port>/<database>?user=<username>&password=<password>`), so you will need to build it from the information provided in the render dashboard.**
3. On the command line, run `gradlew build` in the project root directory.

4. Create a [docker](#) free account; for more information about the Docker environment, check [here](#).
5. Install [Docker Desktop](#)
6. Add the [Dockerfile](#) to the root of the group repository, this file should contain all the information to create the docker image, see our example [here](#).
7. On the command line, do `docker login`.
8. Create the docker image , on the command line, do `docker build -t <docker-username>/img-ls-2526-2-<turma>-g<número-do-grupo> .` (for Mac computers with M1 or M2 include `--platform linux/amd64`)
9. Push the image to docker hub, on the command line, do `docker push <docker-username>/img-ls-2526-2-<turma>-g<número-do-grupo>`
10. On the Render web site, create a new [Web Service](#) with the name `service-ls-2526-2-<turma>-g<número-do-grupo>`
 - Select [Deploy an existing image from a registry](#)
 - Insert Image URL, `docker.io/<docker-username>/img-ls-2526-2-<turma>-g<número-do-grupo>`
 - Name your application and select the "Frankfurt(EU Central)" region
 - Select the free instance type
 - Add a `JDBC_DATABASE_URL` environment variable to your the new Web Service, using the credentials available in render Postgres dashboard (ex.
`jdbc:postgresql://<hostname>.frankfurt-postgres.render.com:<port>/<database>?user=<username>&password=<password>`)
- Click create service and wait for the service creation

Note: The service port must be obtained from the environment variable `PORT`.

To launch application in developer machine:

- `docker run -d -p 9000:8080 --env PORT=8080 --env JDBC_DATABASE_URL="jdbc:postgresql://host.docker.internal/<database>?user=<username>&password=<password>" <docker-username>/img-ls-2526-2-<turma>-g<número-do-grupo>`

Refactoring code

This phase should be used to critically review all code baseline, and refactoring to improve code quality should be a goal. Also, more tests should be produced in order to increase the overall coverage.

Report

The technical report should be updated and/or extended with the relevant technical information. The sections developed in the previous phases can be improved or changed. **There should not be a separate report for each phase.**

Delivery

The completed project must be delivered until 13 June, via the creation of a **1.0.0** tag on the GitHub repository.